1. WORDCOUNT

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class WordCount
{
public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable>
{
private final static IntWritable one = new IntWritable(1);
private Text word = new Text();
public void map(Object key, Text value, Context context) throws IOException, InterruptedException
{
StringTokenizer itr = new StringTokenizer(value.toString());
while (itr.hasMoreTokens())
{
word.set(itr.nextToken());
context.write(word, one);
}
}
}

public static class IntSumReducer extends Reducer<Text,IntWritable,Text,IntWritable>
{
private IntWritable result = new IntWritable();
public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException,
InterruptedException
{
int sum = 0;
for (IntWritable val : values)
{
sum += val.get();
}
result.set(sum);
context.write(key, result);
}
}
```

```
public static void main(String[] args) throws Exception
{
Configuration conf = new Configuration();
Job job = Job.getInstance(conf, "word count");
job.setJarByClass(WordCount.class);
job.setMapperClass(TokenizerMapper.class);
job.setCombinerClass(IntSumReducer.class);
job.setReducerClass(IntSumReducer.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));
System.exit(job.waitForCompletion(true) ? 0 : 1);
}}
```

2.MAXIMUM AVARAGE

```
import java.io.IOException;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

public class mapper extends
Mapper<Object, Text, Text, FloatWritable> {

public void map(Object key, Text value, Context context)
throws IOException, InterruptedException {

String line = value.toString();
String [] a = line.split(" ");
System.out.println(line);
int sum=0;
for(String i:a){
sum += Integer.parseInt(i);
}
float avg = sum/a.length;
System.out.println(avg);
context.write(new Text("maxavg"),new FloatWritable(avg) );

}

}
class MaximumAverageReducer extends Reducer<Text, FloatWritable, Text, FloatWritable> {

public void reduce(Text key, Iterable<FloatWritable> values, Context context)
throws IOException, InterruptedException {

float max=0;
```

```
for (FloatWritable val : values) {
if(val.get()>max){
max=val.get();
}
}
context.write(key, new FloatWritable(max));
}
}
```

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
public class driver {

public static void main(String[] args) throws Exception {
Configuration conf = new Configuration();
Job job = Job.getInstance(conf, "maxaverage");
job.setJarByClass(driver.class);
job.setMapperClass(mapper.class);
job.setReducerClass(MaximumAverageReducer.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(FloatWritable.class);

FileInputFormat.setInputPaths(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

if (!job.waitForCompletion(true))
return;
}

}
```

3.MAXIMUM TEMPERATURE

```
import java.io.IOException;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

public class tmapper extends Mapper<Object, Text, Text, FloatWritable> {

public void map(Object ikey, Text ivalue, Context context)
```

```
throws IOException, InterruptedException {

String line = ivalue.toString();

if (!(line.length()==0)){
String year = line.substring(6, 10);
float avg = Float.parseFloat(line.substring(54,62).trim());
context.write(new Text(year),new FloatWritable(avg));

}

}

}

class MaxTempReducer extends Reducer<Text, FloatWritable, Text, FloatWritable> {

public void reduce(Text key, Iterable<FloatWritable> values, Context context)
throws IOException, InterruptedException {

float max = 0;

for(FloatWritable value:values ){

if(max < value.get()){
max=value.get();

}
}

context.write(key, new FloatWritable(max));
}
}

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class tdriver {

public static void main(String[] args) throws Exception {
Configuration conf = new Configuration();
Job job = Job.getInstance(conf, "maxtemp");
job.setJarByClass(tdriver.class);
job.setMapperClass(tmapper.class);
job.setReducerClass(MaxTempReducer.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(FloatWritable.class);
```

```
FileInputFormat.setInputPaths(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

if (!job.waitForCompletion(true))
return;
}

}
```

4. MATRIX MULTIFICATION

```
import java.io.IOException;
import java.util.*;
import java.util.AbstractMap.SimpleEntry;
import java.util.Map.Entry;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.conf.*;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
public class matrixmatrix {
public static class Map extends Mapper<LongWritable, Text, Text, Text> {
public void map(LongWritable key, Text value, Context context) throws IOException,
InterruptedException {
String line = value.toString();
String[] indicesAndValue = line.split(",");
Text outputKey = new Text();
Text outputValue = new Text();
if (indicesAndValue[0].equals("M")) {
outputKey.set(indicesAndValue[2]);
outputValue.set("M," + indicesAndValue[1] + "," +indicesAndValue[3]);
context.write(outputKey, outputValue);
} else {
outputKey.set(indicesAndValue[1]);
outputValue.set("N," + indicesAndValue[2] + "," +indicesAndValue[3]);
context.write(outputKey, outputValue);
}
}
}
public static class Reduce extends Reducer<Text, Text, Text,Text> {
public void reduce(Text key, Iterable<Text> values,Context context) throws IOException,
InterruptedException {
String[] value;
```

```
ArrayList<Entry<Integer, Float>> listA = new ArrayList<Entry<Integer, Float>>();
ArrayList<Entry<Integer, Float>> listB = new ArrayList<Entry<Integer, Float>>();
for (Text val : values) {
value = val.toString().split(",");
if (value[0].equals("M")) {
listA.add(new SimpleEntry<Integer,
Float>(Integer.parseInt(value[1]), Float.parseFloat(value[2])));
} else {
listB.add(new SimpleEntry<Integer,
Float>(Integer.parseInt(value[1]), Float.parseFloat(value[2])));
}
}
String i;
float a_ij;
String k;
float b_jk;
Text outputValue = new Text();
for (Entry<Integer, Float> a : listA) {
i = Integer.toString(a.getKey());
a_ij = a.getValue();
for (Entry<Integer, Float> b : listB) {
k = Integer.toString(b.getKey());
b_jk = b.getValue();
outputValue.set(i + "," + k + "," + Float.toString(a_ij*b_jk));
context.write(null, outputValue);
}
}
}
}

public static void main(String[] args) throws Exception {
Configuration conf = new Configuration();
Job job = Job.getInstance(conf, "matrixmatrix");
job.setJarByClass(matrixmatrix.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);
job.setMapperClass(Map.class);
job.setReducerClass(Reduce.class);
job.setInputFormatClass(TextInputFormat.class);
job.setOutputFormatClass(TextOutputFormat.class);
FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));
job.waitForCompletion(true);

}
}
```

5.EVEN ODD

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class evenodd {

public static class TokenizeMapper
extends Mapper<Object, Text, Text, Text> {

private static final Text one = new Text();
private Text word = new Text();

@Override
protected void map(Object key, Text value,
Mapper<Object, Text, Text, Text>.Context context)
throws IOException, InterruptedException {
StringTokenizer itr = new StringTokenizer(value.toString());
while (itr.hasMoreTokens()) {
word.set(itr.nextToken());
context.write(word, one);
}
}
}

public static class EvenOddReducer
extends Reducer<Text, Text, Text, Text> {

private Text output = new Text();
private String strNumber = "";
private int number = 0;

@Override
protected void reduce(Text key, Iterable<Text> values,
```

```
Reducer<Text, Text, Text, Text>.Context context)
throws IOException, InterruptedException {

strNumber = key.toString();
number = Integer.parseInt(strNumber);
String result=number % 2 == 0 ? "even" : "odd";

context.write(key, new Text(result));
}
}

public static void main(String[] args) throws Exception {
Configuration conf = new Configuration();
Job job = Job.getInstance(conf, "Even Odd");

job.setJarByClass(evenodd.class);
job.setMapperClass(evenodd.TokenizeMapper.class);
job.setCombinerClass(evenodd.EvenOddReducer.class);
job.setReducerClass(evenodd.EvenOddReducer.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);

FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1);
}

}
```

6.PRIME NUMBER

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class PrimeNumber {

public static class TokenizeMaper
extends Mapper<Object, Text, Text, Text> {
```

```
private Text number = new Text();

private final static Text one = new Text();

@Override
protected void map(Object key, Text value,
Mapper<Object, Text, Text, Text>.Context context)
throws IOException, InterruptedException {
StringTokenizer itr = new StringTokenizer(value.toString());
while (itr.hasMoreTokens()) {
number.set(itr.nextToken());
context.write(number, one);
}
}

}

public static class PrimeReducer
extends Reducer<Text, Text, Text, Text> {

private Text output = new Text();
private String strNumber = "";
private int number = 0;
private boolean isPrime = true;

protected void reduce(Text key, Iterable<Text> values,
Reducer<Text, Text, Text, Text>.Context context)throws IOException, InterruptedException {

isPrime = true;
strNumber = key.toString();
number = Integer.parseInt(strNumber);

// Find if the number is prime or not
for (int i = 2; i <= number/2; i++) {
if(number % i == 0) {
isPrime = false;
break;
}
}

String result=isPrime ? "prime" : "Not Prime";
context.write(key, new Text(result));
}

}

public static void main(String[] args)
throws Exception {
Configuration conf = new Configuration();

Job job = Job.getInstance(conf, "Prime Number");
```

```
job.setJarByClass(PrimeNumber.class);

job.setMapperClass(PrimeNumber.TokenizeMaper.class);
job.setCombinerClass(PrimeNumber.PrimeReducer.class);
job.setReducerClass(PrimeNumber.PrimeReducer.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(Text.class);

FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

System.exit(job.waitForCompletion(true) ? 0 : 1);
}

}
```

7. SALARY AVARAGE

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class mapper{

public static class MapperClass extends
Mapper<LongWritable, Text, Text, FloatWritable> {
public void map(LongWritable key, Text empRecord, Context con)
throws IOException, InterruptedException {
String[] word = empRecord.toString().split("\\t");
String sex = word[3];
try {
Float salary = Float.parseFloat(word[8]);
con.write(new Text(sex), new FloatWritable(salary));
} catch (Exception e) {
e.printStackTrace();
}
```

```
}
}

public static class ReducerClass extends
Reducer<Text, FloatWritable, Text, Text> {
public void reduce(Text key, Iterable<FloatWritable> valueList,
Context con) throws IOException, InterruptedException {
try {
Float total = (float) 0;
int count = 0;
for (FloatWritable var : valueList) {
total += var.get();
System.out.println("reducer " + var.get());
count++;
}
Float avg = (Float) total / count;
String out = "Total: " + total + " :: " + "Average: " + avg;
con.write(key, new Text(out));
} catch (Exception e) {
e.printStackTrace();
}
}
}

public static void main(String[] args) throws Exception{
Configuration conf = new Configuration();

Job job = Job.getInstance(conf, "FindAverageAndTotalSalary");
job.setJarByClass(mapper.class);
job.setMapperClass(MapperClass.class);
job.setReducerClass(ReducerClass.class);
job.setOutputKeyClass(Text.class);
job.setOutputValueClass(FloatWritable.class);
Path p1 = new Path(args[0]);
Path p2 = new Path(args[1]);
FileInputFormat.addInputPath(job, p1);
FileOutputFormat.setOutputPath(job, p2);

System.exit(job.waitForCompletion(true) ? 0 : 1);

}
}
```

8a. WORD COUNT USING PIG

```
lines = LOAD 'demo2.txt' AS (line:chararray);
```

```
words = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) AS word;
grouped = GROUP words BY word;
word_count = FOREACH grouped GENERATE group AS word, COUNT(words) AS count;
DUMP word_count;
```

8b. MAX TEMPARATURE USING PIG

```
data = LOAD 'maxtemp.txt' USING PigStorage(' ') AS (year:int, temperature:int);
grouped = GROUP data BY year;
max_temp = FOREACH grouped GENERATE group AS year, MAX(data.temperature) AS max_temperature;
DUMP max_temp;
```